

Next: [Program Instrumentation Options](#), Previous: [Options for Debugging Your Program](#), Up: [GCC Command Options](#) [\[Contents\]](#)[\[Index\]](#)

3.12 Options That Control Optimization

These options control various sorts of optimizations.

Without any optimization option, the compiler’s goal is to reduce the cost of compilation and to make debugging produce the expected results. Statements are independent: if you stop the program with a breakpoint between statements, you can then assign a new value to any variable or change the program counter to any other statement in the function and get exactly the results you expect from the source code.

Turning on optimization flags makes the compiler attempt to improve the performance and/or code size at the expense of compilation time and possibly the ability to debug the program.

The compiler performs optimization based on the knowledge it has of the program. Compiling multiple files at once to a single output file mode allows the compiler to use information gained from all of the files when compiling each of them.

Not all optimizations are controlled directly by a flag. Only optimizations that have a flag are listed in this section.

Most optimizations are completely disabled at -O0 or if an -O level is not set on the command line, even if individual optimization flags are specified. Similarly, -Og suppresses many optimization passes.

Depending on the target and how GCC was configured, a slightly different set of optimizations may be enabled at each -O level than those listed here. You can invoke GCC with -Q --help=optimizers to find out the exact set of optimizations that are enabled at each level. See [Options Controlling the Kind of Output](#), for examples.

-O

-O1

--optimize

Optimize. Optimizing compilation takes somewhat more time, and a lot more memory for a large function.

With -O, the compiler tries to reduce code size and execution time, without performing any optimizations that take a great deal of compilation time.

-O is the recommended optimization level for large machine-generated code as a sensible balance between time taken to compile and memory use: higher optimization levels perform optimizations with greater algorithmic complexity than at -O.

-O turns on the following optimization flags:

```
-fauto-inc-dec
-fbranch-count-reg
-fcombine-stack-adjustments
-fcompare-elim
-fcprop-registers
-fdce
-fdefer-pop
-fdelayed-branch
-fdse
-fforward-propagate
-fguess-branch-probability
-fif-conversion
-fif-conversion2
-finline-functions-called-once
-fipa-modref
-fipa-profile
-fipa-pure-const
-fipa-reference
-fipa-reference-addressable
-fivopts
-fmerge-constants
-fmove-loop-invariants
-fmove-loop-stores
-fomit-frame-pointer
-freorder-blocks
-fshrink-wrap
-fshrink-wrap-separate
-fsplit-wide-types
-fssa-backprop
-fssa-phiopt
-ftree-bit-ccp
-ftree-ccp
-ftree-ch
-ftree-coalesce-vars
-ftree-copy-prop
-ftree-dce
-ftree-dominator-opts
-ftree-dse
-ftree-forwprop
-ftree-fre
-ftree-phirop
-ftree-pta
-ftree-scev-cprop
-ftree-sink
-ftree-slsr
-ftree-sra
-ftree-ter
-funit-at-a-time
```

-O2

Optimize even more. GCC performs nearly all supported optimizations that do not involve a space-speed tradeoff. As compared to -O, this option increases both compilation time and the performance of the generated code.

-O2 turns on all optimization flags specified by -O1. It also turns on the following optimization flags:

```
-falign-functions -falign-jumps
-falign-labels -falign-loops
-fcaller-saves
-fcode-hoisting
-fcrossjumping
-fcse-follow-jumps -fcse-skip-blocks
-fdelete-null-pointer-checks -fdep-fusion
-fdevirtualize -fdevirtualize-speculatively
-fexpensive-optimizations
-ffinite-loops
-fgcse -fgcse-lm
-fhoist-adjacent-loads
-finline-functions
-finline-small-functions
-findirect-inlining
-fipa-bit-cp -fipa-cp -fipa-icf
-fipa-ra -fipa-sra -fipa-vrp
-fisolate-erroneous-paths-dereference
-flra-remat
-foptimize-crc
-foptimize-sibling-calls
```

```
-foptimize-strlen
-fpartial-inlining
-fpeephole2
-freorder-blocks-algorithm=stc
-freorder-blocks-and-partition  -freorder-functions
-frerun-cse-after-loop
-fschedule-insns  -fschedule-insns2
-fsched-interblock  -fsched-spec
-fspeculatively-call-stored-functions
-fstore-merging
-fstrict-aliasing
-fthread-jumps
-ftime-builtin-call-dce
-ftime-loop-vectorize
-ftime-pre
-ftime-slp-vectorize
-ftime-switch-conversion  -ftime-tail-merge
-ftime-vrp
-fvect-cost-model=very-cheap
```

Please note the warning under `-fgcse` about invoking `-O2` on programs that use computed gotos.

`-O3`

Optimize yet more. `-O3` turns on all optimizations specified by `-O2` and also turns on the following optimization flags:

```
-fgcse-after-reload
-fipa-cp-clone
-floop-interchange
-floop-unroll-and-jam
-fpeel-loops
-fpredictive-commoning
-fsplit-loops
-ftime-loop-distribution
-ftime-partial-pre
-funswitch-loops
-fvect-cost-model=dynamic
-fversion-loops-for-strides
```

`-O0`

Reduce compilation time and make debugging produce the expected results. This is the default.

At `-O0`, GCC completely disables most optimization passes; they are not run even if you explicitly enable them on the command line, or are listed by `-Q --help=optimizers` as being enabled by default. Many optimizations performed by GCC depend on code analysis or canonicalization passes that are enabled by `-O`, and it would not be useful to run individual optimization passes in isolation.

`-Os`

Optimize for size. `-Os` enables all `-O2` optimizations except those that often increase code size:

```
-falign-functions  -falign-jumps
-falign-labels  -falign-loops
-fprefetch-loop-arrays  -freorder-blocks-algorithm=stc
```

It also enables `-finline-functions`, causes the compiler to tune for code size rather than execution speed, and performs further optimizations designed to reduce code size.

`-Ofast`

Disregard strict standards compliance. `-Ofast` enables all `-O3` optimizations. It also enables optimizations that are not valid for all standard-compliant programs. It turns on `-ffast-math`, `-fallow-store-data-races` and the Fortran-specific `-fstack-arrays`, unless `-fmax-stack-var-size` is specified, and `-fno-protect-parens`. It turns off `-fsemantic-interposition`.

`-Og`

Optimize while keeping in mind debugging experience. `-Og` should be the optimization level of choice for the standard edit-compile-debug cycle, offering a reasonable blend of optimization, fast compilation and debugging experience especially for code with a high abstraction penalty. In contrast to `-O0`, this enables `-fvar-tracking-assignments` and `-fvar-tracking` which handle debug information in the prologue and epilogue of functions better than `-O0`.

Like `-O0`, `-Og` completely skips a number of optimization passes so that individual options controlling them have no effect. Otherwise `-Og` enables all `-O1` optimization flags except for those known to greatly interfere with debugging:

```
-fbranch-count-reg  -fdelayed-branch
-fdse  -fif-conversion  -fif-conversion2
-finline-functions-called-once
-fmove-loop-invariants  -fmove-loop-stores  -fssa-phiopt
-ftime-bit-ccp  -ftime-dse  -ftime-pta  -ftime-sra
```

`-Oz`

Optimize aggressively for size rather than speed. This may increase the number of instructions executed if those instructions require fewer bytes to encode. `-Oz` behaves similarly to `-Os` including enabling most `-O2` optimizations.

If you use multiple `-O` options, with or without level numbers, the last such option is the one that is effective.

Options of the form `-fflag` specify machine-independent flags. Most flags have both positive and negative forms; the negative form of `-ffoo` is `-fno-foo`. In the table below, only one of the forms is listed—the one you typically use. You can figure out the other form by either removing ‘no-’ or adding it.

The following options control specific optimizations. They are either activated by `-O` options or are related to ones that are. You can use the following flags in the rare cases when “fine-tuning” of optimizations to be performed is desired.

`-fno-defer-pop`

For machines that must pop arguments after a function call, always pop the arguments as soon as each function returns. At levels `-O1` and higher, `-fdefer-pop` is the default; this allows the compiler to let arguments accumulate on the stack for several function calls and pop them all at once.

`-fforward-propagate`

Perform a forward propagation pass on RTL. The pass tries to combine two instructions and checks if the result can be simplified. If loop unrolling is active, two passes are performed and the second is scheduled after loop unrolling.

This option is enabled by default at optimization levels `-O1`, `-O2`, `-O3`, `-Os`.

`-favoid-store-forwarding`

`-fno-avoid-store-forwarding`

Many CPUs will stall for many cycles when a load partially depends on previous smaller stores. This pass tries to detect such cases and avoid the penalty by changing the order of the load and store and then fixing up the loaded value.

Disabled by default.

-ffp-contract=style

-ffp-contract=off disables floating-point expression contraction. -ffp-contract=fast enables floating-point expression contraction such as forming of fused multiply-add operations if the target has native support for them. -ffp-contract=on enables floating-point expression contraction if allowed by the language standard. This is implemented for C and C++, where it enables contraction within one expression, but not across different statements.

The default is -ffp-contract=off for C in a standards compliant mode (-std=c11 or similar), -ffp-contract=fast otherwise.

-ffp-int-builtin-inexact

Allow the built-in functions ceil, floor, round and trunc, and their float and long double variants, to generate code that raises the “inexact” floating-point exception for noninteger arguments. ISO C99 and C11 allow these functions to raise the “inexact” exception, but ISO/IEC TS 18661-1:2014, the C bindings to IEEE 754-2008, as integrated into ISO C23, does not allow these functions to do so.

The default is -fno-fp-int-builtin-inexact, disallowing the exception to be raised, unless C17 or an earlier C standard is selected. This option does nothing unless -ftrapping-math is in effect.

Even if -fno-fp-int-builtin-inexact is used, if the functions generate a call to a library function then the “inexact” exception may be raised if the library implementation does not follow TS 18661.

-fomit-frame-pointer

Omit the frame pointer in functions that don’t need one. This avoids the instructions to save, set up and restore the frame pointer; on many targets it also makes an extra register available.

On some targets this flag has no effect because the standard calling sequence always uses a frame pointer, so it cannot be omitted.

Note that -fno-omit-frame-pointer doesn’t guarantee the frame pointer is used in all functions. Several targets always omit the frame pointer in leaf functions.

Enabled by default at -O1 and higher.

-foptimize-crc

Detect loops calculating CRC (performing polynomial long division) and replace them with a faster implementation. Detect 8, 16, 32, and 64 bit CRC, with a constant polynomial without the leading 1 bit, for both bit-forward and bit-reversed cases. If the target supports a CRC instruction and the polynomial used in the source code matches the polynomial used in the CRC instruction, generate that CRC instruction. Otherwise, if the target supports a carry-less-multiplication instruction, generate CRC using it; otherwise generate table-based CRC.

Enabled by default at -O2 and higher.

-foptimize-sibling-calls

Optimize sibling and tail recursive calls.

Enabled at levels -O2, -O3, -Os.

-foptimize-strlen

Optimize various standard C string functions (e.g. strlen, strchr or strcpy) and their _FORTIFY_SOURCE counterparts into faster alternatives.

Enabled at levels -O2, -O3.

-finline-atomics

-fno-inline-atomics

Inline ‘__atomic’ operations when a lock-free instruction sequence is available. This optimization is enabled by default.

-finline-stringops[=fn]

Expand memory and string operations (for now, only memset) inline, even when the length is variable or big enough as to require looping. This is most useful along with -ffreestanding and -fno-builtin.

In some circumstances, it enables the compiler to generate code that takes advantage of known alignment and length multipliers, but even then it may be less efficient than optimized runtime implementations, and grow code size so much that even a less performant but shared implementation runs faster due to better use of code caches. This option is disabled by default.

-fno-inline

Do not expand any functions inline apart from those marked with the always_inline attribute. This is the default when not optimizing.

Single functions can be exempted from inlining by marking them with the noinline attribute.

-finline-small-functions

Integrate functions into their callers when their body is smaller than expected function call code (so overall size of program gets smaller). The compiler heuristically decides which functions are simple enough to be worth integrating in this way. This inlining applies to all functions, even those not declared inline.

Enabled at levels -O2, -O3, -Os.

-findirect-inlining

Inline also indirect calls that are discovered to be known at compile time thanks to previous inlining. This option has any effect only when inlining itself is turned on by the -finline-functions or -finline-small-functions options.

Enabled at levels -O2, -O3, -Os.

-finline-functions

Consider all functions for inlining, even if they are not declared inline. The compiler heuristically decides which functions are worth integrating in this way.

If all calls to a given function are integrated, and the function is declared static, then the function is normally not output as assembler code in its own right.

Enabled at levels -O2, -O3, -Os. Also enabled by -fprofile-use and -fauto-profile.

-finline-functions-called-once

Consider all static functions called once for inlining into their caller even if they are not marked inline. If a call to a given function is integrated, then the function is not output as assembler code in its own right.

Enabled at levels -O1, -O2, -O3 and -Os, but not -Og.

-fearly-inlining

Inline functions marked by always_inline and functions whose body seems smaller than the function call overhead early before doing -fprofile-generate instrumentation and real inlining pass. Doing so makes profiling significantly cheaper and usually inlining faster on programs having large chains of nested wrapper functions.

Enabled by default.

-fipa-sra

	Perform interprocedural scalar replacement of aggregates, removal of unused parameters and replacement of parameters passed by reference by parameters passed by value. Enabled at levels -O2, -O3 and -Os.
-finline-limit= <i>n</i>	By default, GCC limits the size of functions that can be inlined. This flag allows coarse control of this limit. <i>n</i> is the size of functions that can be inlined in number of pseudo instructions. Inlining is actually controlled by a number of internal parameters, which are documented in the GCC Internals manual and should not normally be set directly. <i>Note:</i> there may be no value to -finline-limit that results in default behavior. <i>Note:</i> pseudo instruction represents, in this particular context, an abstract measurement of function’s size. In no way does it represent a count of assembly instructions and as such its exact meaning might change from one release to an another.
-fno-keep-inline-dllexport	This is a more fine-grained version of -fkeep-inline-functions, which applies only to functions that are declared using the dllexport attribute or declspec. See Common Attributes.
-fkeep-inline-functions	In C, emit static functions that are declared inline into the object file, even if the function has been inlined into all of its callers. This switch does not affect functions using the extern inline extension in GNU C90. In C++, emit any and all inline functions into the object file.
-fkeep-static-functions	Emit static functions into the object file, even if the function is never used.
-fkeep-static-consts	Emit variables declared static const when optimization isn’t turned on, even if the variables aren’t referenced. GCC enables this option by default. If you want to force the compiler to check if a variable is referenced, regardless of whether or not optimization is turned on, use the -fno-keep-static-consts option.
-fmerge-constants	Attempt to merge identical constants (string constants and floating-point constants) across compilation units. This option is the default for optimized compilation if the assembler and linker support it. Use -fno-merge-constants to inhibit this behavior. Enabled at levels -O1, -O2, -O3, -Os.
-fmerge-all-constants	Attempt to merge identical constants and identical variables. This option implies -fmerge-constants. In addition to -fmerge-constants this considers e.g. even constant initialized arrays or initialized constant variables with integral or floating-point types. Languages like C or C++ require each variable, including multiple instances of the same variable in recursive calls, to have distinct locations, so using this option results in non-conforming behavior.
-fmodulo-sched	Perform swing modulo scheduling immediately before the first scheduling pass. This pass looks at innermost loops and reorders their instructions by overlapping different iterations.
-fmodulo-sched-allow-regmoves	Perform more aggressive SMS-based modulo scheduling with register moves allowed. By setting this flag certain anti-dependences edges are deleted, which triggers the generation of reg-moves based on the life-range analysis. This option is effective only with -fmodulo-sched enabled.
-fno-branch-count-reg	Disable the optimization pass that scans for opportunities to use “decrement and branch” instructions on a count register instead of instruction sequences that decrement a register, compare it against zero, and then branch based upon the result. This option is only meaningful on architectures that support such instructions, which include x86, PowerPC, IA-64 and S/390. Note that the -fno-branch-count-reg option doesn’t remove the decrement and branch instructions from the generated instruction stream introduced by other optimization passes. The default is -fbranch-count-reg at -O1 and higher, except for -Og.
-fno-function-cse	Do not put function addresses in registers; make each instruction that calls a constant function contain the function’s address explicitly. This option results in less efficient code, but some strange hacks that alter the assembler output may be confused by the optimizations performed when this option is not used. The default is -ffunction-cse.
-ffuse-ops-with-volatile-access	Allow limited optimization of operations with volatile memory access when doing so does not change the semantics outlined in See When is a Volatile Object Accessed?. The default is -ffuse-ops-with-volatile-access
-fno-zero-initialized-in-bss	If the target supports a BSS section, GCC by default puts variables that are initialized to zero into BSS. This can save space in the resulting code. This option turns off this behavior because some programs explicitly rely on variables going to the data section—e.g., so that the resulting executable can find the beginning of that section and/or make assumptions based on that. The default is -fzero-initialized-in-bss except in Ada.
-fthread-jumps	Perform optimizations that check to see if a jump branches to a location where another comparison subsumed by the first is found. If so, the first branch is redirected to either the destination of the second branch or a point immediately following it, depending on whether the condition is known to be true or false. Enabled at levels -O1, -O2, -O3, -Os.
-fsplit-wide-types	When using a type that occupies multiple registers, such as long long on a 32-bit system, split the registers apart and allocate them independently. This normally generates better code for those types, but may make debugging more difficult. Enabled at levels -O1, -O2, -O3, -Os.

-fsplit-wide-types-early

Fully split wide types early, instead of very late. This option has no effect unless -fsplit-wide-types is turned on.

This is the default on some targets.

-fcse-follow-jumps

In common subexpression elimination (CSE), scan through jump instructions when the target of the jump is not reached by any other path. For example, when CSE encounters an if statement with an else clause, CSE follows the jump when the condition tested is false.

Enabled at levels -O2, -O3, -Os.

-fcse-skip-blocks

This is similar to -fcse-follow-jumps, but causes CSE to follow jumps that conditionally skip over blocks. When CSE encounters a simple if statement with no else clause, -fcse-skip-blocks causes CSE to follow the jump around the body of the if.

Enabled at levels -O2, -O3, -Os.

-frerun-cse-after-loop

Re-run common subexpression elimination after loop optimizations are performed.

Enabled at levels -O2, -O3, -Os.

-fgcse

Perform a global common subexpression elimination pass. This pass also performs global constant and copy propagation.

Note: When compiling a program using computed gotos, a GCC extension, you may get better run-time performance if you disable the global common subexpression elimination pass by adding -fno-gcse to the command line.

Enabled at levels -O2, -O3, -Os.

-fgcse-lm

When -fgcse-lm is enabled, global common subexpression elimination attempts to move loads that are only killed by stores into themselves. This allows a loop containing a load/store sequence to be changed to a load outside the loop, and a copy/store within the loop.

Enabled by default when -fgcse is enabled.

-fgcse-sm

When -fgcse-sm is enabled, a store motion pass is run after global common subexpression elimination. This pass attempts to move stores out of loops. When used in conjunction with -fgcse-lm, loops containing a load/store sequence can be changed to a load before the loop and a store after the loop.

Not enabled at any optimization level.

-fgcse-las

When -fgcse-las is enabled, the global common subexpression elimination pass eliminates redundant loads that come after stores to the same memory location (both partial and full redundancies).

Not enabled at any optimization level.

-fgcse-after-reload

When -fgcse-after-reload is enabled, a redundant load elimination pass is performed after reload. The purpose of this pass is to clean up redundant spilling.

Enabled by -O3, -fprofile-use and -fauto-profile.

-faggressive-loop-optimizations

This option tells the loop optimizer to use language constraints to derive bounds for the number of iterations of a loop. This assumes that loop code does not invoke undefined behavior by for example causing signed integer overflows or out-of-bound array accesses. The bounds for the number of iterations of a loop are used to guide loop unrolling and peeling and loop exit test optimizations. This option is enabled by default.

-funconstrained-commons

This option tells the compiler that variables declared in common blocks (e.g. Fortran) may later be overridden with longer trailing arrays. This prevents certain optimizations that depend on knowing the array bounds.

-fcrossjumping

Perform cross-jumping transformation. This transformation unifies equivalent code and saves code size. The resulting code may or may not perform better than without cross-jumping.

Enabled at levels -O2, -O3, -Os.

-fauto-inc-dec

Combine increments or decrements of addresses with memory accesses. This pass is always skipped on architectures that do not have instructions to support this. Enabled by default at -O1 and higher on architectures that support this.

-fdce

Perform dead code elimination (DCE) on RTL. Enabled by default at -O1 and higher.

-fdse

Perform dead store elimination (DSE) on RTL. Enabled by default at -O1 and higher.

-fif-conversion

Attempt to transform conditional jumps into branch-less equivalents. This includes use of conditional moves, min, max, set flags and abs instructions, and some tricks doable by standard arithmetics. The use of conditional execution on chips where it is available is controlled by -fif-conversion2.

Enabled at levels -O1, -O2, -O3, -Os, but not with -Og.

-fif-conversion2

Use conditional execution (where available) to transform conditional jumps into branch-less equivalents.

Enabled at levels -O1, -O2, -O3, -Os, but not with -Og.

-fdeclone-ctor-dtor

The C++ ABI requires multiple entry points for constructors and destructors: one for a base subobject, one for a complete object, and one for a virtual destructor that calls operator delete afterwards. For a hierarchy with virtual bases, the base and complete variants are clones, which means two copies of the function. With this option, the base and complete variants are changed to be thunks that call a common implementation.

Enabled by -Os.
-fdelete-null-pointer-checks Assume that programs cannot safely dereference null pointers, and that no code or data element resides at address zero. This option enables simple constant folding optimizations at all optimization levels. In addition, other optimization passes in GCC use this flag to control global dataflow analyses that eliminate useless checks for null pointers; these assume that a memory access to address zero always results in a trap, so that if a pointer is checked after it has already been dereferenced, it cannot be null. Note however that in some environments this assumption is not true. Use -fno-delete-null-pointer-checks to disable this optimization for programs that depend on that behavior. This option is enabled by default on most targets. On AVR and MSP430, this option is completely disabled. Passes that use the dataflow information are enabled independently at different optimization levels.
-fdevirtualize Attempt to convert calls to virtual functions to direct calls. This is done both within a procedure and interprocedurally as part of indirect inlining (-findirect-inlining) and interprocedural constant propagation (-fipa-cp). Enabled at levels -O2, -O3, -Os.
-fdevirtualize-speculatively Attempt to convert calls to virtual functions to speculative direct calls. Based on the analysis of the type inheritance graph, determine for a given call the set of likely targets. If the set is small, preferably of size 1, change the call into a conditional deciding between direct and indirect calls. The speculative calls enable more optimizations, such as inlining. When they seem useless after further optimization, they are converted back into original form.
-fdevirtualize-at-ltrans Stream extra information needed for aggressive devirtualization when running the link-time optimizer in local transformation mode. This option enables more devirtualization but significantly increases the size of streamed data. For this reason it is disabled by default.
-fexpensive-optimizations Perform a number of minor optimizations that are relatively expensive. Enabled at levels -O2, -O3, -Os.
-fext-dce -fno-ext-dce Perform dead code elimination on zero and sign extensions, with special dataflow analysis.
-free Attempt to remove redundant extension instructions. This is especially helpful for the x86-64 architecture, which implicitly zero-extends in 64-bit registers after writing to their lower 32-bit half. Enabled for Alpha, AArch64, LoongArch, PowerPC, RISC-V, SPARC, h83000 and x86 at levels -O2, -O3, -Os.
-fno-lifetime-dse In C++ the value of an object is only affected by changes within its lifetime: when the constructor begins, the object has an indeterminate value, and any changes during the lifetime of the object are dead when the object is destroyed. Normally dead store elimination will take advantage of this; if your code relies on the value of the object storage persisting beyond the lifetime of the object, you can use this flag to disable this optimization. To preserve stores before the constructor starts (e.g. because your operator new clears the object storage) but still treat the object as dead after the destructor, you can use -flifetime-dse=1. The default behavior can be explicitly selected with -flifetime-dse=2. -flifetime-dse=0 is equivalent to -fno-lifetime-dse.
-flive-range-shrinkage Attempt to decrease register pressure through register live range shrinkage. This is helpful for fast processors with small or moderate size register sets.
-fira-algorithm=<i>algorithm</i> Use the specified coloring algorithm for the integrated register allocator. The <i>algorithm</i> argument can be ‘priority’, which specifies Chow’s priority coloring, or ‘cb’, which specifies Chaitin-Briggs coloring. Chaitin-Briggs coloring is not implemented for all architectures, but for those targets that do support it, it is the default because it generates better code.
-fira-region=<i>region</i> Use specified regions for the integrated register allocator. The <i>region</i> argument should be one of the following: ‘all’ Use all loops as register allocation regions. This can give the best results for machines with a small and/or irregular register set. ‘mixed’ Use all loops except for loops with small register pressure as the regions. This value usually gives the best results in most cases and for most architectures, and is enabled by default when compiling with optimization for speed (-O, -O2, ...). ‘one’ Use all functions as a single region. This typically results in the smallest code size, and is enabled by default for -Os or -O0.
-fira-hoist-pressure Use IRA to evaluate register pressure in the code hoisting pass for decisions to hoist expressions. This option usually results in smaller code, but it can slow the compiler down. This option is enabled at level -Os for all targets.
-fira-loop-pressure Use IRA to evaluate register pressure in loops for decisions to move loop invariants. This option usually results in generation of faster and smaller code on machines with large register files (>= 32 registers), but it can slow the compiler down. This option is enabled at level -O3 for some targets.
-fno-ira-share-save-slots Disable sharing of stack slots used for saving call-used hard registers living through a call. Each hard register gets a separate stack slot, and as a result function stack frames are larger.
-fno-ira-share-spill-slots Disable sharing of stack slots allocated for pseudo-registers. Each pseudo-register that does not get a hard register gets a separate stack slot, and as a result function stack frames are larger.
-flra-remat

	Enable CFG-sensitive rematerialization in LRA. Instead of loading values of spilled pseudos, LRA tries to rematerialize (recalculate) values if it is profitable. Enabled at levels -02, -03, -0s.
-fdelayed-branch	If supported for the target machine, attempt to reorder instructions to exploit instruction slots available after delayed branch instructions. Enabled at levels -01, -02, -03, -0s, but not at -0g.
-fschedule-insns	If supported for the target machine, attempt to reorder instructions to eliminate execution stalls due to required data being unavailable. This helps machines that have slow floating point or memory load instructions by allowing other instructions to be issued until the result of the load or floating-point instruction is required. Conventionally enabled at optimization levels -02 and -03. However, many targets override this behavior. For example, on x86, it is disabled at all levels, while on AArch64, it is enabled only at -03.
-fschedule-insns2	Similar to -fschedule-insns, but requests an additional pass of instruction scheduling after register allocation has been done. This is especially useful on machines with a relatively small number of registers and where memory load instructions take more than one cycle. Enabled at levels -02, -03, -0s.
-fno-sched-interblock	Disable instruction scheduling across basic blocks, which is normally enabled when scheduling before register allocation, i.e. with -fschedule-insns or at -02 or higher.
-fno-sched-spec	Disable speculative motion of non-load instructions, which is normally enabled when scheduling before register allocation, i.e. with -fschedule-insns or at -02 or higher.
-fsched-pressure	Enable register pressure sensitive insn scheduling before register allocation. This only makes sense when scheduling before register allocation is enabled, i.e. with -fschedule-insns or at -02 or higher. Usage of this option can improve the generated code and decrease its size by preventing register pressure increase above the number of available hard registers and subsequent spills in register allocation.
-fsched-spec-load	Allow speculative motion of some load instructions. This only makes sense when scheduling before register allocation, i.e. with -fschedule-insns or at -02 or higher.
-fsched-spec-load-dangerous	Allow speculative motion of more load instructions. This only makes sense when scheduling before register allocation, i.e. with -fschedule-insns or at -02 or higher.
-fsched-stalled-insns -fsched-stalled-insns= <i>n</i>	Define how many insns (if any) can be moved prematurely from the queue of stalled insns into the ready list during the second scheduling pass. -fno-sched-stalled-insns means that no insns are moved prematurely, -fsched-stalled-insns=0 means there is no limit on how many queued insns can be moved prematurely. -fsched-stalled-insns without a value is equivalent to -fsched-stalled-insns=1.
-fsched-stalled-insns-dep -fsched-stalled-insns-dep= <i>n</i>	Define how many insn groups (cycles) are examined for a dependency on a stalled insn that is a candidate for premature removal from the queue of stalled insns. This has an effect only during the second scheduling pass, and only if -fsched-stalled-insns is used. -fno-sched-stalled-insns-dep is equivalent to -fsched-stalled-insns-dep=0. -fsched-stalled-insns-dep without a value is equivalent to -fsched-stalled-insns-dep=1.
-fsched2-use-superblocks	When scheduling after register allocation, use superblock scheduling. This allows motion across basic block boundaries, resulting in faster schedules. This option is experimental, as not all machine descriptions used by GCC model the CPU closely enough to avoid unreliable results from the algorithm. This only makes sense when scheduling after register allocation, i.e. with -fschedule-insns2 or at -02 or higher.
-fsched-group-heuristic	Enable the group heuristic in the scheduler. This heuristic favors the instruction that belongs to a schedule group. This is enabled by default when scheduling is enabled, i.e. with -fschedule-insns or -fschedule-insns2 or at -02 or higher.
-fsched-critical-path-heuristic	Enable the critical-path heuristic in the scheduler. This heuristic favors instructions on the critical path. This is enabled by default when scheduling is enabled, i.e. with -fschedule-insns or -fschedule-insns2 or at -02 or higher.
-fsched-spec-insn-heuristic	Enable the speculative instruction heuristic in the scheduler. This heuristic favors speculative instructions with greater dependency weakness. This is enabled by default when scheduling is enabled, i.e. with -fschedule-insns or -fschedule-insns2 or at -02 or higher.
-fsched-rank-heuristic	Enable the rank heuristic in the scheduler. This heuristic favors the instruction belonging to a basic block with greater size or frequency. This is enabled by default when scheduling is enabled, i.e. with -fschedule-insns or -fschedule-insns2 or at -02 or higher.
-fsched-last-insn-heuristic	Enable the last-instruction heuristic in the scheduler. This heuristic favors the instruction that is less dependent on the last instruction scheduled. This is enabled by default when scheduling is enabled, i.e. with -fschedule-insns or -fschedule-insns2 or at -02 or higher.
-fsched-dep-count-heuristic	Enable the dependent-count heuristic in the scheduler. This heuristic favors the instruction that has more instructions depending on it. This is enabled by default when scheduling is enabled, i.e. with -fschedule-insns or -fschedule-insns2 or at -02 or higher.
-fspeculatively-call-stored-functions	Attempt to convert indirect calls of function pointers to pointers loaded from a structure field if all visible stores to that field store just a single candidate. When doing so, turn the call into a conditional deciding between the direct call and the original indirect one. These speculative calls often enable more optimizations, such as inlining. When they seem useless after further optimization, they are converted back into original form.
-freschedule-modulo-scheduled-loops	

	Modulo scheduling is performed before traditional scheduling. If a loop is modulo scheduled, later scheduling passes may change its schedule. Use this option to control that behavior.
-fselective-scheduling	
	Schedule instructions using selective scheduling algorithm. Selective scheduling runs instead of the first scheduler pass.
-fselective-scheduling2	
	Schedule instructions using selective scheduling algorithm. Selective scheduling runs instead of the second scheduler pass.
-fsel-sched-pipelining	
	Enable software pipelining of innermost loops during selective scheduling. This option has no effect unless one of -fselective-scheduling or -fselective-scheduling2 is turned on.
-fsel-sched-pipelining-outer-loops	
	When pipelining loops during selective scheduling, also pipeline outer loops. This option has no effect unless -fsel-sched-pipelining is turned on.
-fsemantic-interposition	
	Some object formats, like ELF, allow interposing of symbols by the dynamic linker. This means that for symbols exported from the DSO, the compiler cannot perform interprocedural propagation, inlining and other optimizations in anticipation that the function or variable in question may change. While this feature is useful, for example, to rewrite memory allocation functions by a debugging implementation, it is expensive in the terms of code quality. With -fno-semantic-interposition the compiler assumes that if interposition happens for functions the overwriting function will have precisely the same semantics (and side effects). Similarly if interposition happens for variables, the constructor of the variable will be the same. The flag has no effect for functions explicitly declared inline (where it is never allowed for interposition to change semantics) and for symbols explicitly declared weak.
-fshrink-wrap	
	Emit function prologues only before parts of the function that need it, rather than at the top of the function. This flag is enabled by default at -O and higher.
-fshrink-wrap-separate	
	Shrink-wrap separate parts of the prologue and epilogue separately, so that those parts are only executed when needed. This option is on by default, but has no effect unless -fshrink-wrap is also turned on and the target supports this.
-fcaller-saves	
	Enable allocation of values to registers that are clobbered by function calls, by emitting extra instructions to save and restore the registers around such calls. Such allocation is done only when it seems to result in better code.
	This option is always enabled by default on certain machines, usually those which have no call-preserved registers to use instead.
	Enabled at levels -O2, -O3, -Os.
-fcombine-stack-adjustments	
	Tracks stack adjustments (pushes and pops) and stack memory references and then tries to find ways to combine them.
	Enabled by default at -O1 and higher.
-fipa-ra	
	Use caller save registers for allocation if those registers are not used by any called function. In that case it is not necessary to save and restore them around calls. This is only possible if called functions are part of same compilation unit as current function and they are compiled before it.
	Enabled at levels -O2, -O3, -Os, however the option is disabled if generated code will be instrumented for profiling (-p, or -pg) or if callee’s register usage cannot be known exactly (this happens on targets that do not expose prologues and epilogues in RTL).
-fconserve-stack	
	Attempt to minimize stack usage. The compiler attempts to use less stack space, even if that makes the program slower. This option implies setting the large-stack-frame parameter to 100 and the large-stack-frame-growth parameter to 400.
-ftree-reassoc	
	Perform reassociation on trees. This flag is enabled by default at -O1 and higher.
-fcode-hoisting	
	Perform code hoisting. Code hoisting tries to move the evaluation of expressions executed on all paths to the function exit as early as possible. This is especially useful as a code size optimization, but it often helps for code speed as well. This flag is enabled by default at -O2 and higher.
-ftree-pre	
	Perform partial redundancy elimination (PRE) on trees. This flag is enabled by default at -O2 and -O3.
-ftree-partial-pre	
	Make partial redundancy elimination (PRE) more aggressive. This flag is enabled by default at -O3.
-ftree-forwprop	
	Perform forward propagation on trees. This flag is enabled by default at -O1 and higher.
-ftree-fre	
	Perform full redundancy elimination (FRE) on trees. The difference between FRE and PRE is that FRE only considers expressions that are computed on all paths leading to the redundant computation. This analysis is faster than PRE, though it exposes fewer redundancies. This flag is enabled by default at -O1 and higher.
-ftree-phirop	
	Perform hoisting of loads from conditional pointers on trees. This pass is enabled by default at -O1 and higher.
-fhoist-adjacent-loads	
	Speculatively hoist loads from both branches of an if-then-else if the loads are from adjacent locations in the same structure and the target architecture has a conditional move instruction. This flag is enabled by default at -O2 and higher.
-ftree-copy-prop	
	Perform copy propagation on trees. This pass eliminates unnecessary copy operations. This flag is enabled by default at -O1 and higher.
-fipa-pure-const	
	Discover which functions are pure or constant. Enabled by default at -O1 and higher.
-fipa-reference	
	Discover which static variables do not escape the compilation unit. Enabled by default at -O1 and higher.

-fipa-reference-addressable Discover read-only, write-only and non-addressable static variables. Enabled by default at -O1 and higher.	
-fipa-reorder-for-locality Group call chains close together in the binary layout to improve code locality and minimize jump distances between frequently called functions. Unlike -freorder-functions this pass considers the call chains between functions and groups them together, rather than grouping all hot/normal/cold/never-executed functions into separate sections. Unlike -fprofile-reorder-functions it aims to improve code locality throughout the runtime of the program rather than focusing on program startup. This option is incompatible with an explicit -flto-partition= option since it enforces a custom partitioning scheme. If using this option it is recommended to also use profile feedback, but this option is not enabled by default otherwise.	
-fipa-stack-alignment Reduce stack alignment on call sites if possible. Enabled by default.	
-fipa-pta Perform interprocedural pointer analysis and interprocedural modification and reference analysis. This option can cause excessive memory and compile-time usage on large compilation units. It is not enabled by default at any optimization level.	
-fipa-profile Perform interprocedural profile propagation. The functions called only from cold functions are marked as cold. Also functions executed once (such as cold, noreturn, static constructors or destructors) are identified. Cold functions and loop less parts of functions executed once are then optimized for size. Enabled by default at -O1 and higher.	
-fipa-modref Perform interprocedural mod/ref analysis. This optimization analyzes the side effects of functions (memory locations that are modified or referenced) and enables better optimization across the function call boundary. This flag is enabled by default at -O1 and higher.	
-fipa-cp Perform interprocedural constant propagation. This optimization analyzes the program to determine when values passed to functions are constants and then optimizes accordingly. This optimization can substantially increase performance if the application has constants passed to functions. This flag is enabled by default at -O2, -Os and -O3. It is also enabled by -fprofile-use and -fauto-profile.	
-fipa-cp-clone Perform function cloning to make interprocedural constant propagation stronger. When enabled, interprocedural constant propagation performs function cloning when externally visible function can be called with constant arguments. Because this optimization can create multiple copies of functions, it may significantly increase code size. This flag is enabled by default at -O3. It is also enabled by -fprofile-use and -fauto-profile.	
-fipa-bit-cp When enabled, perform interprocedural bitwise constant propagation. This flag is enabled by default at -O2 and by -fprofile-use and -fauto-profile. It requires that -fipa-cp is enabled.	
-fipa-vrp When enabled, perform interprocedural propagation of value ranges. This flag is enabled by default at -O2. It requires that -fipa-cp is enabled.	
-fipa-icf-functions -fipa-icf-variables -fipa-icf Perform Identical Code Folding for functions (-fipa-icf-functions), read-only variables (-fipa-icf-variables), or both (-fipa-icf). The optimization reduces code size and may disturb unwind stacks by replacing a function by an equivalent one with a different name. The optimization works more effectively with link-time optimization enabled. Although the behavior is similar to the Gold Linker’s ICF optimization, GCC ICF works on different levels and thus the optimizations are not same - there are equivalences that are found only by GCC and equivalences found only by Gold. -fipa-icf is enabled by default at -O2 and -Os.	
-flate-combine-instructions Enable two instruction combination passes that run relatively late in the compilation process. One of the passes runs before register allocation and the other after register allocation. The main aim of the passes is to substitute definitions into all uses. Most targets enable this flag by default at -O2 and -Os.	
-flive-patching=Level Control GCC’s optimizations to produce output suitable for live-patching. If the compiler’s optimization uses a function’s body or information extracted from its body to optimize/change another function, the latter is called an impacted function of the former. If a function is patched, its impacted functions should be patched too. The impacted functions are determined by the compiler’s interprocedural optimizations. For example, a caller is impacted when inlining a function into its caller, cloning a function and changing its caller to call this new clone, or extracting a function’s pureness/constness information to optimize its direct or indirect callers, etc. Usually, the more IPA optimizations enabled, the larger the number of impacted functions for each function. In order to control the number of impacted functions and more easily compute the list of impacted function, IPA optimizations can be partially enabled at two different levels. The level argument should be one of the following: ‘inline-clone’ Only enable inlining and cloning optimizations, which includes inlining, cloning, interprocedural scalar replacement of aggregates and partial inlining. As a result, when patching a function, all its callers and its clones’ callers are impacted, therefore need to be patched as well. -flive-patching=inline-clone disables the following optimization flags: -fwhole-program -fipa-pta -fipa-reference -fipa-ra -fipa-icf -fipa-icf-functions -fipa-icf-variables -fipa-bit-cp -fipa-vrp -fipa-pure-const -fipa-reference-addressable -fipa-stack-alignment -fipa-modref ‘inline-only-static’ Only enable inlining of static functions. As a result, when patching a static function, all its callers are impacted and so need to be patched as well. In addition to all the flags that -flive-patching=inline-clone disables, -flive-patching=inline-only-static disables the following additional optimization flags:	

-fipa-cp-clone -fipa-sra -fpartial-inlining -fipa-cp

When -flive-patching is specified without any value, the default value is *inline-clone*.

This flag is disabled by default.

Note that -flive-patching is not supported with link-time optimization (-flto).

-fisolate-erroneous-paths-dereference

Detect paths that trigger erroneous or undefined behavior due to dereferencing a null pointer (with -fdelete-null-pointer-checks enabled) or a division by zero. Isolate those paths from the main control flow and turn the statement with erroneous or undefined behavior into a trap. This flag is enabled by default at -02 and higher.

-fisolate-erroneous-paths-attribute

Detect paths that trigger erroneous or undefined behavior due to a null value being used in a way forbidden by a returns_nonnull or nonnull attribute. Isolate those paths from the main control flow and turn the statement with erroneous or undefined behavior into a trap. This is not currently enabled, but may be enabled by -02 in the future.

-ftree-sink

Perform forward store motion on trees. This flag is enabled by default at -01 and higher.

-ftree-bit-ccp

Perform sparse conditional bit constant propagation on trees and propagate pointer alignment information. This pass only operates on local scalar variables and is enabled by default at -01 and higher, except for -0g. It requires that -ftree-ccp is enabled.

-ftree-ccp

Perform sparse conditional constant propagation (CCP) on trees. This pass only operates on local scalar variables and is enabled by default at -01 and higher.

-fssa-backprop

Propagate information about uses of a value up the definition chain in order to simplify the definitions. For example, this pass strips sign operations if the sign of a value never matters. The flag is enabled by default at -01 and higher.

-fssa-phiopt

Perform pattern matching on SSA PHI nodes to optimize conditional code. This pass is enabled by default at -01 and higher, except for -0g.

-ftree-switch-conversion

Perform conversion of simple initializations in a switch to initializations from a scalar array. This flag is enabled by default at -02 and higher.

-ftree-tail-merge

Look for identical code sequences. When found, replace one with a jump to the other. This optimization is known as tail merging or cross jumping. This flag is enabled by default at -02 and higher. The compilation time in this pass can be limited using max-tail-merge-comparisons parameter and max-tail-merge-iterations parameter.

-ftree-cselim

Perform conditional store elimination on trees. This flag is enabled by default at -01 and higher on targets that have conditional move instructions.

-ftree-dce

Perform dead code elimination (DCE) on trees. This flag is enabled by default at -01 and higher.

-ftree-builtin-call-dce

Perform conditional dead code elimination (DCE) for calls to built-in functions that may set errno but are otherwise free of side effects. This flag is enabled by default at -02 and higher if -0s is not also specified.

-ffinite-loops

Assume that a loop with an exit will eventually take the exit and not loop indefinitely. This allows the compiler to remove loops that otherwise have no side-effects, not considering eventual endless looping as such.

This option is enabled by default at -02 for C++ with -std=c++11 or higher.

-ftree-dominator-opts

Perform a variety of simple scalar cleanups (constant/copy propagation, redundancy elimination, range propagation and expression simplification) based on a dominator tree traversal. This also performs jump threading (to reduce jumps to jumps). This flag is enabled by default at -01 and higher.

-ftree-dse

Perform dead store elimination (DSE) on trees. A dead store is a store into a memory location that is later overwritten by another store without any intervening loads. In this case the earlier store can be deleted. This flag is enabled by default at -01 and higher.

-ftree-ch

Perform loop header copying on trees. This is beneficial since it increases effectiveness of code motion optimizations. It also saves one jump. This flag is enabled by default at -01 and higher. It is not enabled for -0s, since it usually increases code size.

-ftree-loop-optimize

Perform loop optimizations on trees. This flag is enabled by default at -01 and higher.

-ftree-loop-linear

-floop-strip-mine

-floop-block

Perform loop nest optimizations. Same as -floop-nest-optimize. To use this code transformation, GCC has to be configured with --with-isl to enable the Graphite loop transformation infrastructure.

-fgraphite-identity

Enable the identity transformation for graphite. For every SCoP we generate the polyhedral representation and transform it back to gimple. Using -fgraphite-identity we can check the costs or benefits of the GIMPLE -> GRAPHITE -> GIMPLE transformation. Some minimal optimizations are also performed by the code generator isl, like index splitting and dead code elimination in loops.

-floop-nest-optimize

Enable the isl based loop nest optimizer. This is a generic loop nest optimizer based on the Pluto optimization algorithms. It calculates a loop structure optimized for data-locality and parallelism. This option is experimental.

-floop-parallelize-all

Use the Graphite data dependence analysis to identify loops that can be parallelized. Parallelize all the loops that can be analyzed to not contain loop carried dependencies without checking that it is profitable to parallelize the loops.

-ftree-coalesce-vars

While transforming the program out of the SSA representation, attempt to reduce copying by coalescing versions of different user-defined variables, instead of just compiler temporaries. This may severely limit the ability to debug an optimized program compiled with -fno-var-tracking-assignments. In the negated form, this flag prevents SSA coalescing of user variables. This option is enabled by default if optimization is enabled, and it does very little otherwise.

-ftree-loop-distribution

Perform loop distribution. This flag can improve cache performance on big loop bodies and allow further loop optimizations, like parallelization or vectorization, to take place. For example, the loop

```
DO I = 1, N
  A(I) = B(I) + C
  D(I) = E(I) * F
ENDDO
```

is transformed to

```
DO I = 1, N
  A(I) = B(I) + C
ENDDO
DO I = 1, N
  D(I) = E(I) * F
ENDDO
```

This flag is enabled by default at -O3. It is also enabled by -fprofile-use and -fauto-profile.

-ftree-loop-distribute-patterns

Perform loop distribution of patterns that can be code generated with calls to a library. This flag is enabled by default at -O2 and higher, and by -fprofile-use and -fauto-profile.

This pass distributes the initialization loops and generates a call to memset zero. For example, the loop

```
DO I = 1, N
  A(I) = 0
  B(I) = A(I) + I
ENDDO
```

is transformed to

```
DO I = 1, N
  A(I) = 0
ENDDO
DO I = 1, N
  B(I) = A(I) + I
ENDDO
```

and the initialization loop is transformed into a call to memset zero.

-floop-interchange

Perform loop interchange outside of graphite. This flag can improve cache performance on loop nest and allow further loop optimizations, like vectorization, to take place. For example, the loop

```
for (int i = 0; i < N; i++)
  for (int j = 0; j < N; j++)
    for (int k = 0; k < N; k++)
      c[i][j] = c[i][j] + a[i][k]*b[k][j];
```

is transformed to

```
for (int i = 0; i < N; i++)
  for (int k = 0; k < N; k++)
    for (int j = 0; j < N; j++)
      c[i][j] = c[i][j] + a[i][k]*b[k][j];
```

This flag is enabled by default at -O3. It is also enabled by -fprofile-use and -fauto-profile.

-floop-unroll-and-jam

Apply unroll and jam transformations on feasible loops. In a loop nest this unrolls the outer loop by some factor and fuses the resulting multiple inner loops. This flag is enabled by default at -O3. It is also enabled by -fprofile-use and -fauto-profile.

-ftree-loop-im

Perform loop invariant motion on trees. This pass moves only invariants that are hard to handle at RTL level (function calls, operations that expand to nontrivial sequences of insns). With -funswitch-loops it also moves operands of conditions that are invariant out of the loop, so that we can use just trivial invariantness analysis in loop unswitching. The pass also includes store motion.

-ftree-loop-ivcanon

Create a canonical counter for number of iterations in loops for which determining number of iterations requires complicated analysis. Later optimizations then may determine the number easily. Useful especially in connection with unrolling.

-ftree-scev-cprop

Perform final value replacement. If a variable is modified in a loop in such a way that its value when exiting the loop can be determined using only its initial value and the number of loop iterations, replace uses of the final value by such a computation, provided it is sufficiently cheap. This reduces data dependencies and may allow further simplifications. Enabled by default at -O1 and higher.

-fivopts

Perform induction variable optimizations (strength reduction, induction variable merging and induction variable elimination) on trees. Enabled by default at -O1 and higher.

-ftree-parallelize-loops

-ftree-parallelize-loops=*n*

Parallelize loops, i.e., split their iteration space to run in multiple threads. This is only possible for loops whose iterations are independent and can be arbitrarily reordered. The optimization is only profitable on multiprocessor machines, for loops that are CPU-intensive, rather than constrained e.g. by memory bandwidth. This option implies -pthread, and thus is only supported on targets that have support for -pthread.

When a positive value n is specified, the number of threads is fixed at compile time and cannot be changed after compilation. The compiler generates “`#pragma omp parallel num_threads( $n$ )`”.

When used without `= $n$` (i.e., `-ftree-parallelize-loops`), the number of threads is determined at program execution time via the `OMP_NUM_THREADS` environment variable. If `OMP_NUM_THREADS` is not set, the OpenMP runtime automatically detects the number of available processors and uses that value. This enables creating binaries that adapt to different hardware configurations without recompilation.

`-ftree-pta`

Perform function-local points-to analysis on trees. This flag is enabled by default at `-O1` and higher, except for `-Og`.

`-ftree-sra`

Perform scalar replacement of aggregates. This pass replaces structure references with scalars to prevent committing structures to memory too early. This flag is enabled by default at `-O1` and higher, except for `-Og`.

`-fstore-merging`

Perform merging of narrow stores to consecutive memory addresses. This pass merges contiguous stores of immediate values narrower than a word into fewer wider stores to reduce the number of instructions. This is enabled by default at `-O2` and higher as well as `-Os`.

`-ftree-ter`

Perform temporary expression replacement during the SSA->normal phase. Single use/single def temporaries are replaced at their use location with their defining expression. This results in non-GIMPLE code, but gives the expanders much more complex trees to work on resulting in better RTL generation. This is enabled by default at `-O1` and higher.

`-ftree-slsr`

Perform straight-line strength reduction on trees. This recognizes related expressions involving multiplications and replaces them by less expensive calculations when possible. This is enabled by default at `-O1` and higher.

`-ftree-vectorize`

Perform vectorization on trees. This flag enables `-ftree-loop-vectorize` and `-ftree-slp-vectorize` if not explicitly specified.

`-ftree-loop-vectorize`

Perform loop vectorization on trees. This flag is enabled by default at `-O2` and by `-ftree-vectorize`, `-fprofile-use`, and `-fauto-profile`.

`-ftree-slp-vectorize`

Perform basic block vectorization on trees. This flag is enabled by default at `-O2` and by `-ftree-vectorize`, `-fprofile-use`, and `-fauto-profile`.

`-ftrivial-auto-var-init=choice`

Initialize automatic variables or temporary objects with either a pattern or with zeroes to increase the security and predictability of a program by preventing uninitialized memory disclosure and use. GCC still considers an automatic variable that doesn’t have an explicit initializer as uninitialized, `-Wuninitialized` and `-Wanalyzer-use-of-uninitialized-value` will still report warning messages on such automatic variables or temporary objects and the compiler will perform optimization as if the variable were uninitialized. With this option, GCC will also initialize any padding of automatic variables or temporary objects that have structure or union types to zeroes. However, the current implementation cannot initialize automatic variables whose initialization is bypassed through `switch` or `goto` statement. Using `-Wtrivial-auto-var-init` to report all such cases.

The three values of *choice* are:

- ‘uninitialized’ doesn’t initialize any automatic variables.
- ‘pattern’ Initialize automatic variables with values which will likely transform logic bugs into crashes down the line, are easily recognized in a crash dump and without being values that programmers can rely on for useful program semantics. The current value is byte-repeatable pattern with byte “0xFE”. The values used for pattern initialization might be changed in the future.
- ‘zero’ Initialize automatic variables with zeroes.

The default is ‘uninitialized’ except for C++26, in which case if `-ftrivial-auto-var-init=` is not specified at all automatic variables or temporary objects are zero initialized, but zero initialization of padding bits does not happen.

Note that the initializer values, whether ‘zero’ or ‘pattern’, refer to data representation (in memory or machine registers), rather than to their interpretation as numerical values. This distinction may be important in languages that support types with biases or implicit multipliers, and with such extensions as ‘hardbool’ (see [Common Attributes](#)). For example, a variable that uses 8 bits to represent (biased) quantities in the range `160..400` will be initialized with the bit patterns `0x00` or `0xFE`, depending on *choice*, whether or not these representations stand for values in that range, and even if they do, the interpretation of the value held by the variable will depend on the bias. A ‘hardbool’ variable that uses say `0x5A` and `0xA5` for `false` and `true`, respectively, will trap with either ‘choice’ of trivial initializer, i.e., ‘zero’ initialization will not convert to the representation for `false`, even if it would for a static variable of the same type. This means the initializer pattern doesn’t generally depend on the type of the initialized variable. One notable exception is that (non-hardened) boolean variables that fit in registers are initialized with `false` (zero), even when ‘pattern’ is requested.

You can control this behavior for a specific variable by using the variable attribute `uninitialized` standard attribute (see [Common Attributes](#)) or the C++26 `[[indeterminate]]`.

`-fvect-cost-model=model`

Alter the cost model used for vectorization. The *model* argument should be one of ‘unlimited’, ‘dynamic’, ‘cheap’ or ‘very-cheap’. With the ‘unlimited’ model the vectorized code-path is assumed to be profitable while with the ‘dynamic’ model a runtime check guards the vectorized code-path to enable it only for iteration counts that will likely execute faster than when executing the original scalar loop. The ‘cheap’ model disables vectorization of loops where doing so would be cost prohibitive for example due to required runtime checks for data dependence or alignment but otherwise is equal to the ‘dynamic’ model. The ‘very-cheap’ model disables vectorization of loops when any runtime check for data dependence or alignment is required, it also disables vectorization of epilogue loops but otherwise is equal to the ‘cheap’ model.

The default cost model depends on other optimization flags and is either ‘dynamic’ or ‘cheap’.

`-fsimd-cost-model=model`

Alter the cost model used for vectorization of loops marked with the OpenMP `simd` directive. The *model* argument should be one of ‘unlimited’, ‘dynamic’, ‘cheap’. All values of *model* have the same meaning as described in `-fvect-cost-model` and by default a cost model defined with `-fvect-cost-model` is used.

`-ftree-rrp`

Perform Value Range Propagation on trees. This is similar to the constant propagation pass, but instead of values, ranges of values are propagated. This allows the optimizers to remove unnecessary range checks like array bound checks and null pointer checks. This is enabled by default at `-O2` and higher. Null pointer check elimination is only done if `-fdelete-null-pointer-checks` is enabled.

`-fsplit-paths`

This option is deprecated and does nothing. It is accepted only for backward compatibility.

`-fsplit-ivs-in-unroller`

Enables expression of values of induction variables in later iterations of the unrolled loop using the value in the first iteration. This breaks long dependency chains, thus improving efficiency of the scheduling passes.

A combination of `-fweb` and CSE is often sufficient to obtain the same effect. However, that is not reliable in cases where the loop body is more complicated than a single basic block. It also does not work at all on some architectures due to restrictions in the CSE pass.

This optimization is enabled by default.

-fvariable-expansion-in-unroller

With this option, the compiler creates multiple copies of some local variables when unrolling a loop, which can result in superior code.

This optimization is enabled by default for PowerPC targets, but disabled by default otherwise.

-fpartial-inlining

Inline parts of functions. This option has any effect only when inlining itself is turned on by the `-finline-functions` or `-finline-small-functions` options.

Enabled at levels `-O2`, `-O3`, `-Os`.

-fpredictive-commoning

Perform predictive commoning optimization, i.e., reusing computations (especially memory loads and stores) performed in previous iterations of loops.

This option is enabled at level `-O3`. It is also enabled by `-fprofile-use` and `-fauto-profile`.

-fprefetch-loop-arrays

If supported by the target machine, generate instructions to prefetch memory to improve the performance of loops that access large arrays.

This option may generate better or worse code; results are highly dependent on the structure of loops within the source code.

Disabled at level `-Os`.

-fno-printf-return-value

Do not substitute constants for known return value of formatted output functions such as `sprintf`, `snprintf`, `vsprintf`, and `vsnprintf` (but not `printf` or `fprintf`). This transformation allows GCC to optimize or even eliminate branches based on the known return value of these functions called with arguments that are either constant, or whose values are known to be in a range that makes determining the exact return value possible. For example, when `-fprintf-return-value` is in effect, both the branch and the body of the `if` statement (but not the call to `snprint`) can be optimized away when `i` is a 32-bit or smaller integer because the return value is guaranteed to be at most 8.

```
char buf[9];
if (snprintf (buf, "%08x", i) >= sizeof buf)
...
```

The `-fprintf-return-value` option relies on other optimizations and yields best results with `-O2` and above. It works in tandem with the `-wformat-overflow` and `-wformat-truncation` options. The `-fprintf-return-value` option is enabled by default.

-fno-peephole

-fno-peephole2

Disable any machine-specific peephole optimizations. The difference between `-fno-peephole` and `-fno-peephole2` is in how they are implemented in the compiler; some targets use one, some use the other, a few use both.

`-fpeephole` is enabled by default. `-fpeephole2` enabled at levels `-O2`, `-O3`, `-Os`.

-fno-guess-branch-probability

Do not guess branch probabilities using heuristics.

GCC uses heuristics to guess branch probabilities if they are not provided by profiling feedback (`-fprofile-arcs`). These heuristics are based on the control flow graph. If some branch probabilities are specified by `__builtin_expect`, then the heuristics are used to guess branch probabilities for the rest of the control flow graph, taking the `__builtin_expect` info into account. The interactions between the heuristics and `__builtin_expect` can be complex, and in some cases, it may be useful to disable the heuristics so that the effects of `__builtin_expect` are easier to understand.

It is also possible to specify expected probability of the expression with `__builtin_expect_with_probability` built-in function.

The default is `-fguess-branch-probability` at levels `-O`, `-O2`, `-O3`, `-Os`.

-freorder-blocks

Reorder basic blocks in the compiled function in order to reduce number of taken branches and improve code locality.

Enabled at levels `-O1`, `-O2`, `-O3`, `-Os`.

-freorder-blocks-algorithm=*algorithm*

Use the specified algorithm for basic block reordering. The *algorithm* argument can be `'simple'`, which does not increase code size (except sometimes due to secondary effects like alignment), or `'stc'`, the “software trace cache” algorithm, which tries to put all often executed code together, minimizing the number of branches executed by making extra copies of code.

The default is `'simple'` at levels `-O1`, `-Os`, and `'stc'` at levels `-O2`, `-O3`.

-freorder-blocks-and-partition

In addition to reordering basic blocks in the compiled function, in order to reduce number of taken branches, partitions hot and cold basic blocks into separate sections of the assembly and `.o` files, to improve paging and cache locality performance.

This optimization is automatically turned off in the presence of exception handling or unwind tables (on targets using `setjump/longjump` or target specific scheme), for linkonce sections, for functions with a user-defined section attribute and on any architecture that does not support named sections. When `-fsplit-stack` is used this option is not enabled by default (to avoid linker errors), but may be enabled explicitly (if using a working linker).

Enabled for x86 at levels `-O2`, `-O3`, `-Os`.

-freorder-functions

Reorder functions in the object file in order to improve code locality. Unlike `-fipa-reorder-for-locality` this option prioritises grouping all functions within a category (hot/normal/cold/never-executed) together. This is implemented by using special subsections `.text.hot` for most frequently executed functions and `.text.unlikely` for unlikely executed functions. Reordering is done by the linker so object file format must support named sections and linker must place them in a reasonable way.

This option isn’t effective unless you either provide profile feedback (see `-fprofile-arcs` for details) or manually annotate functions with `hot` or `cold` attributes (see [Common Attributes](#)).

Enabled at levels `-O2`, `-O3`, `-Os`.

-fstrict-aliasing

Allow the compiler to assume the strictest aliasing rules applicable to the language being compiled. For C (and C++), this activates optimizations based on the type of expressions. In particular, accessing an object of one type via an expression of a different type is not allowed, unless the types are *compatible types*, differ only in signedness or qualifiers, or the expression has a character type. Accessing scalar objects via a corresponding vector type is also allowed.

For example, an `unsigned int` can alias an `int`, but not a `void*` or a `double`. A character type may alias any other type.

Pay special attention to code like this:

```
union a_union {
    int i;
    double d;
};

int f() {
    union a_union t;
    t.d = 3.0;
    return t.i;
}
```

The practice of reading from a different union member than the one most recently written to (called “type-punning”) is common. Even with `-fstrict-aliasing`, type-punning is allowed in C, provided the memory is accessed through the union type. In ISO C++, type-punning through a union type is undefined behavior, but GCC supports it as an extension. So, the code above works as expected. See [Structures, Unions, Enumerations, and Bit-Fields](#). However, this code might not:

```
int f() {
    union a_union t;
    int* ip;
    t.d = 3.0;
    ip = &t.i;
    return *ip;
}
```

Similarly, access by taking the address, casting the resulting pointer and dereferencing the result has undefined behavior, even if the cast uses a union type, e.g.:

```
int f() {
    double d = 3.0;
    return ((union a_union *) &d)->i;
}
```

The `-fstrict-aliasing` option is enabled at levels `-O2`, `-O3`, `-Os`.

`-fipa-strict-aliasing`

Controls whether rules of `-fstrict-aliasing` are applied across function boundaries. Note that if multiple functions gets inlined into a single function the memory accesses are no longer considered to be crossing a function boundary.

The `-fipa-strict-aliasing` option is enabled by default and is effective only in combination with `-fstrict-aliasing`.

- `-falign-functions`
- `-falign-functions=n`
- `-falign-functions=n:m`
- `-falign-functions=n:m:n2`
- `-falign-functions=n:m:n2:m2`

Align the start of functions to the next power-of-two greater than or equal to *n*, skipping up to *m*-1 bytes. This ensures that at least the first *m* bytes of the function can be fetched by the CPU without crossing an *n*-byte alignment boundary. This is an optimization of code performance and alignment is ignored for functions considered cold. If alignment is required for all functions, use `-fmin-function-alignment`.

If *m* is not specified, it defaults to *n*.

Examples: `-falign-functions=32` aligns functions to the next 32-byte boundary, `-falign-functions=24` aligns to the next 32-byte boundary only if this can be done by skipping 23 bytes or less, `-falign-functions=32:7` aligns to the next 32-byte boundary only if this can be done by skipping 6 bytes or less.

The second pair of *n2*:*m2* values allows you to specify a secondary alignment: `-falign-functions=64:7:32:3` aligns to the next 64-byte boundary if this can be done by skipping 6 bytes or less, otherwise aligns to the next 32-byte boundary if this can be done by skipping 2 bytes or less. If *m2* is not specified, it defaults to *n2*.

Some assemblers only support this flag when *n* is a power of two; in that case, it is rounded up.

`-fno-align-functions` and `-falign-functions=1` are equivalent and mean that functions are not aligned.

If *n* is not specified or is zero, use a machine-dependent default. The maximum allowed *n* option value is 65536.

Enabled at levels `-O2`, `-O3`.

`-flimit-function-alignment`

If this option is enabled, the compiler tries to avoid unnecessarily overaligning functions. It attempts to instruct the assembler to align by the amount specified by `-falign-functions`, but not to skip more bytes than the size of the function.

- `-falign-labels`
- `-falign-labels=n`
- `-falign-labels=n:m`
- `-falign-labels=n:m:n2`
- `-falign-labels=n:m:n2:m2`

Align all branch targets to a power-of-two boundary.

Parameters of this option are analogous to the `-falign-functions` option. `-fno-align-labels` and `-falign-labels=1` are equivalent and mean that labels are not aligned.

If `-falign-loops` or `-falign-jumps` are applicable and are greater than this value, then their values are used instead.

If *n* is not specified or is zero, use a machine-dependent default which is very likely to be ‘1’, meaning no alignment. The maximum allowed *n* option value is 65536.

Enabled at levels `-O2`, `-O3`.

- `-falign-loops`
- `-falign-loops=n`
- `-falign-loops=n:m`
- `-falign-loops=n:m:n2`
- `-falign-loops=n:m:n2:m2`

Align loops to a power-of-two boundary. If the loops are executed many times, this makes up for any execution of the dummy padding instructions. This is an optimization of code performance and alignment is ignored for loops considered cold.

If `-falign-labels` is greater than this value, then its value is used instead.

Parameters of this option are analogous to the `-falign-functions` option. `-fno-align-loops` and `-falign-loops=1` are equivalent and mean that loops are not aligned. The maximum allowed *n* option value is 65536.

If *n* is not specified or is zero, use a machine-dependent default.

Enabled at levels `-O2`, `-O3`.

`-falign-jumps`

`-falign-jumps=n`
`-falign-jumps=n:m`
`-falign-jumps=n:m:n2`
`-falign-jumps=n:m:n2:m2`

Align branch targets to a power-of-two boundary, for branch targets where the targets can only be reached by jumping. In this case, no dummy operations need be executed. This is an optimization of code performance and alignment is ignored for jumps considered cold.

If `-falign-labels` is greater than this value, then its value is used instead.

Parameters of this option are analogous to the `-falign-functions` option. `-fno-align-jumps` and `-falign-jumps=1` are equivalent and mean that loops are not aligned.

If *n* is not specified or is zero, use a machine-dependent default. The maximum allowed *n* option value is 65536.

Enabled at levels `-O2`, `-O3`.

`-fmin-function-alignment`

Specify minimal alignment of functions to the next power-of-two greater than or equal to *n*. Unlike `-falign-functions` this alignment is applied also to all functions (even those considered cold). The alignment is also not affected by `-flimit-function-alignment`

`-fno-allocation-dce`

Do not remove unused C++ allocations (using operator `new` and operator `delete`) in dead code elimination.

See also `-fmalloc-dce`.

`-fallow-store-data-races`

Allow the compiler to perform optimizations that may introduce new data races on stores, without proving that the variable cannot be concurrently accessed by other threads. Does not affect optimization of local data. It is safe to use this option if it is known that global data will not be accessed by multiple threads.

Examples of optimizations enabled by `-fallow-store-data-races` include hoisting or if-conversions that may cause a value that was already in memory to be re-written with that same value. Such re-writing is safe in a single threaded context but may be unsafe in a multi-threaded context. Note that on some processors, if-conversions may be required in order to enable vectorization.

Enabled at level `-Ofast`.

`-funit-at-a-time`

This option is left for compatibility reasons. `-funit-at-a-time` has no effect, while `-fno-unit-at-a-time` implies `-fno-toplevel-reorder` and `-fno-section-anchors`.

Enabled by default.

`-fno-toplevel-reorder`

Do not reorder top-level functions, variables, and `asm` statements. Output them in the same order that they appear in the input file. When this option is used, unreferenced static variables are not removed. This option is intended to support existing code that relies on a particular ordering. For new code, it is better to use attributes when possible.

`-ftoplevel-reorder` is the default at `-O1` and higher, and also at `-O0` if `-fsection-anchors` is explicitly requested. Additionally `-fno-toplevel-reorder` implies `-fno-section-anchors`.

`-funreachable-traps`

With this option, the compiler turns calls to `__builtin_unreachable` into traps, instead of using them for optimization. This also affects any such calls implicitly generated by the compiler.

This option has the same effect as `-fsanitize=unreachable` `-fsanitize-trap=unreachable`, but does not affect the values of those options. If `-fsanitize=unreachable` is enabled, that option takes priority over this one.

This option is enabled by default at `-O0` and `-Og`.

`-fweb`

Constructs webs as commonly used for register allocation purposes and assign each web individual pseudo register. This allows the register allocation pass to operate on pseudos directly, but also strengthens several other optimization passes, such as CSE, loop optimizer and trivial dead code remover. It can, however, make debugging impossible, since variables no longer stay in a “home register”.

Enabled by default with `-funroll-loops`.

`-fwhole-program`

Assume that the current compilation unit represents the whole program being compiled. All public functions and variables with the exception of `main` and those merged by attribute `externally_visible` become static functions and in effect are optimized more aggressively by interprocedural optimizers.

With `-flto` this option has a limited use. In most cases the precise list of symbols used or exported from the binary is known the resolution info passed to the link-time optimizer by the linker plugin. It is still useful if no linker plugin is used or during incremental link step when final code is produced (with `-flto` `-flinker-output=nolto-rel`).

`-flto[=n]`

This option runs the standard link-time optimizer. When invoked with source code, it generates GIMPLE (one of GCC’s internal representations) and writes it to special ELF sections in the object file. When the object files are linked together, all the function bodies are read from these ELF sections and instantiated as if they had been part of the same translation unit.

To use the link-time optimizer, `-flto` and optimization options should be specified at compile time and during the final link. It is recommended that you compile all the files participating in the same link with the same options and also specify those options at link time. For example:

```
gcc -c -O2 -flto foo.c
gcc -c -O2 -flto bar.c
gcc -o myprog -flto -O2 foo.o bar.o
```

The first two invocations to GCC save a bytecode representation of GIMPLE into special ELF sections inside `foo.o` and `bar.o`. The final invocation reads the GIMPLE bytecode from `foo.o` and `bar.o`, merges the two files into a single internal image, and compiles the result as usual. Since both `foo.o` and `bar.o` are merged into a single image, this causes all the interprocedural analyses and optimizations in GCC to work across the two files as if they were a single one. This means, for example, that the inliner is able to inline functions in `bar.o` into functions in `foo.o` and vice-versa.

Another (simpler) way to enable link-time optimization is:

```
gcc -o myprog -flto -O2 foo.c bar.c
```

The above generates bytecode for `foo.c` and `bar.c`, merges them together into a single GIMPLE representation and optimizes them as usual to produce `myprog`.

The important thing to keep in mind is that to enable link-time optimizations you need to use the GCC driver to perform the link step. GCC automatically performs link-time optimization if any of the objects involved were compiled with the `-flto` command-line option. You can always override the automatic decision to do

link-time optimization by passing `-fno-lto` to the link command.

To make whole-program optimization effective, it is necessary to make certain assumptions. The compiler needs to know what functions and variables can be accessed by libraries and runtime outside of the link-time optimized unit. When supported by the linker, the linker plugin (see `-fuse-linker-plugin`) passes information to the compiler about used and externally visible symbols. When the linker plugin is not available, `-fwhole-program` should be used to allow the compiler to make these assumptions, which leads to more aggressive optimization decisions.

When a file is compiled with `-flto` without `-fuse-linker-plugin`, the generated object file is larger than a regular object file because it contains GIMPLE bytecodes and the usual final code (see `-ffat-lto-objects`). This means that object files with LTO information can be linked as normal object files; if `-fno-lto` is passed to the linker, no interprocedural optimizations are applied. Note that when `-fno-fat-lto-objects` is enabled the compile stage is faster but you cannot perform a regular, non-LTO link on them.

When producing the final binary, GCC only applies link-time optimizations to those files that contain bytecode. Therefore, you can mix and match object files and libraries with GIMPLE bytecodes and final object code. GCC automatically selects which files to optimize in LTO mode and which files to link without further processing.

Generally, options specified at link time override those specified at compile time, although in some cases GCC attempts to infer link-time options from the settings used to compile the input files.

If you do not specify an optimization level option `-O` at link time, then GCC uses the highest optimization level used when compiling the object files. Note that it is generally ineffective to specify an optimization level option only at link time and not at compile time, for two reasons. First, compiling without optimization suppresses compiler passes that gather information needed for effective optimization at link time. Second, some early optimization passes can be performed only at compile time and not at link time.

There are some code generation flags preserved by GCC when generating bytecodes, as they need to be used during the final link. Currently, the following options and their settings are taken from the first object file that explicitly specifies them: `-fcommon`, `-fexceptions`, `-fnon-call-exceptions`, `-fgnu-tm` and all the `-m` target flags.

The following options `-fPIC`, `-fpic`, `-fpie` and `-fPIE` are combined based on the following scheme:

```
-fPIC + -fpic = -fpic
-fPIC + -fno-pic = -fno-pic
-fpic/-fPIC + (no option) = (no option)
-fPIC + -fPIE = -fPIE
-fpic + -fPIE = -fpie
-fPIC/-fpic + -fpie = -fpie
```

Certain ABI-changing flags are required to match in all compilation units, and trying to override this at link time with a conflicting value is ignored. This includes options such as `-freg-struct-return` and `-fpcc-struct-return`.

Other options such as `-ffp-contract`, `-fno-strict-overflow`, `-fwrapv`, `-fno-trapv` or `-fno-strict-aliasing` are passed through to the link stage and merged conservatively for conflicting translation units. Specifically `-fno-strict-overflow`, `-fwrapv` and `-fno-trapv` take precedence; and for example `-ffp-contract=off` takes precedence over `-ffp-contract=fast`. You can override them at link time.

Diagnostic options such as `-wstringop-overflow` are passed through to the link stage and their setting matches that of the compile-step at function granularity. Note that this matters only for diagnostics emitted during optimization. Note that code transforms such as inlining can lead to warnings being enabled or disabled for regions if code not consistent with the setting at compile time.

When you need to pass options to the assembler via `-wa` or `-xassembler` make sure to either compile such translation units with `-fno-lto` or consistently use the same assembler options on all translation units. You can alternatively also specify assembler options at LTO link time.

To enable debug info generation you need to supply `-g` at compile time. If any of the input files at link time were built with debug info generation enabled the link will enable debug info generation as well. Any elaborate debug info settings like the dwarf level `-gdwarf-5` need to be explicitly repeated at the linker command line and mixing different settings in different translation units is discouraged.

If LTO encounters objects with C linkage declared with incompatible types in separate translation units to be linked together (undefined behavior according to ISO C99 6.2.7), a non-fatal diagnostic may be issued. The behavior is still undefined at run time. Similar diagnostics may be raised for other languages.

Another feature of LTO is that it is possible to apply interprocedural optimizations on files written in different languages:

```
gcc -c -flto foo.c
g++ -c -flto bar.cc
gfortran -c -flto baz.f90
g++ -o myprog -flto -O3 foo.o bar.o baz.o -lgfortran
```

Notice that the final link is done with `g++` to get the C++ runtime libraries and `-lgfortran` is added to get the Fortran runtime libraries. In general, when mixing languages in LTO mode, you should use the same link command options as when mixing languages in a regular (non-LTO) compilation.

If object files containing GIMPLE bytecode are stored in a library archive, say `libfoo.a`, it is possible to extract and use them in an LTO link if you are using a linker with plugin support. To create static libraries suitable for LTO, use `gcc-ar` and `gcc-ranlib` instead of `ar` and `ranlib`; to show the symbols of object files with GIMPLE bytecode, use `gcc-nm`. Those commands require that `ar`, `ranlib` and `nm` have been compiled with plugin support. At link time, use the flag `-fuse-linker-plugin` to ensure that the library participates in the LTO optimization process:

```
gcc -o myprog -O2 -flto -fuse-linker-plugin a.o b.o -lfoo
```

With the linker plugin enabled, the linker extracts the needed GIMPLE files from `libfoo.a` and passes them on to the running GCC to make them part of the aggregated GIMPLE image to be optimized.

If you are not using a linker with plugin support and/or do not enable the linker plugin, then the objects inside `libfoo.a` are extracted and linked as usual, but they do not participate in the LTO optimization process. In order to make a static library suitable for both LTO optimization and usual linkage, compile its object files with `-flto -ffat-lto-objects`.

Link-time optimizations do not require the presence of the whole program to operate. If the program does not require any symbols to be exported, it is possible to combine `-flto` and `-fwhole-program` to allow the interprocedural optimizers to use more aggressive assumptions which may lead to improved optimization opportunities. Use of `-fwhole-program` is not needed when linker plugin is active (see `-fuse-linker-plugin`).

The current implementation of LTO makes no attempt to generate bytecode that is portable between different types of hosts. The bytecode files are versioned and there is a strict version check, so bytecode files generated in one version of GCC do not work with an older or newer version of GCC.

Link-time optimization does not work well with generation of debugging information on systems other than those using a combination of ELF and DWARF.

If you specify the optional *n*, the optimization and code generation done at link time is executed in parallel using *n* parallel jobs by utilizing an installed `make` program. The environment variable `MAKE` may be used to override the program used.

You can also specify `-flto=jobserver` to use GNU `make`'s job server mode to determine the number of parallel jobs. This is useful when the Makefile calling GCC is already executing in parallel. You must prepend a '+' to the command recipe in the parent Makefile for this to work. This option likely only works if `MAKE` is GNU `make`. Even without the option value, GCC tries to automatically detect a running GNU `make`'s job server.

Use `-flto=auto` to use GNU `make`'s job server, if available, or otherwise fall back to autodetection of the number of CPU threads present in your system.

`-flto-partition=alg`

Specify the partitioning algorithm used by the link-time optimizer. The value is either `'1to1'` to specify a partitioning mirroring the original source files or `'balanced'` to specify partitioning into equally sized chunks (whenever possible) or `'max'` to create new partition for every symbol where possible or `'cache'` to balance chunk sizes while keeping related symbols together for better caching in incremental LTO. Specifying `'none'` as an algorithm disables partitioning and

streaming completely. The default value is ‘balanced’. While ‘1to1’ can be used as an workaround for various code ordering issues, the ‘max’ partitioning is intended for internal testing only. The value ‘one’ specifies that exactly one partition should be used while the value ‘none’ bypasses partitioning and executes the link-time optimization step directly from the WPA phase.

`-flto-incremental=`*path*

Enable incremental LTO, with its cache in given existing directory. Can significantly shorten edit-compile cycles with LTO.

When used with LTO (`-flto`), the output of translation units inside LTO is cached. Cached translation units are likely to be encountered again when recompiling with small code changes, leading to recompile time reduction.

Multiple GCC instances can use the same cache in parallel.

`-flto-incremental-cache-size=`*n*

Specifies number of cache entries in incremental LTO after which to prune old entries. This is a soft limit, temporarily there may be more entries.

`-flto-compression-level=`*n*

This option specifies the level of compression used for intermediate language written to LTO object files, and is only meaningful in conjunction with LTO mode (`-flto`). GCC currently supports two LTO compression algorithms. For `zstd`, valid values are 0 (no compression) to 19 (maximum compression), while `zlib` supports values from 0 to 9. Values outside this range are clamped to either minimum or maximum of the supported values. If the option is not given, a default balanced compression setting is used.

`-flto-toplevel-asm-heuristics`

Enables heuristics to find symbols used in top-level basic `asm`. This will restrict link-time optimizations that could cause renaming or deletion of such symbols which would result in missing symbol errors by linker.

This flag is intended for projects that have not converted to using top-level extended `asm` (see [Extended Asm - Assembler Instructions with C Expression Operands](#)), which specify the usage directly without any false positives.

The heuristics are simple and do not parse the assembly. The heuristics scan through top-level assembly for all possible identifiers; if an identifier is found among declared symbols, the symbol will be marked to restrict link-time optimizations. Static symbols disable more optimizations. Identifiers followed by ‘:’ disable more optimizations as well, because they might be a locally defined symbol in assembly, even when the declaration is marked ‘extern’.

`-fuse-linker-plugin`

Enables the use of a linker plugin during link-time optimization. This option relies on plugin support in the linker, which is available in gold or in GNU ld 2.21 or newer.

This option enables the extraction of object files with GIMPLE bytecode out of library archives. This improves the quality of optimization by exposing more code to the link-time optimizer. This information specifies what symbols can be accessed externally (by non-LTO object or during dynamic linking). Resulting code quality improvements on binaries (and shared libraries that use hidden visibility) are similar to `-fwhole-program`. See `-flto` for a description of the effect of this flag and how to use it.

This option is enabled by default when LTO support in GCC is enabled and GCC was configured for use with a linker supporting plugins (GNU ld 2.21 or newer or gold).

`-ffat-lto-objects`

Fat LTO objects are object files that contain both the intermediate language and the object code. This makes them usable for both LTO linking and normal linking. This option is effective only when compiling with `-flto` and is ignored at link time.

`-fno-fat-lto-objects` improves compilation time over plain LTO, but requires the complete toolchain to be aware of LTO. It requires a linker with linker plugin support for basic functionality. Additionally, `nm`, `ar` and `ranlib` need to support linker plugins to allow a full-featured build environment (capable of building static libraries etc). GCC provides the `gcc-ar`, `gcc-nm`, `gcc-ranlib` wrappers to pass the right options to these tools. With non fat LTO makefiles need to be modified to use them.

Note that modern binutils provide plugin auto-load mechanism. Installing the linker plugin into `$libdir/bfd-plugins` has the same effect as usage of the command wrappers (`gcc-ar`, `gcc-nm` and `gcc-ranlib`).

The default is `-fno-fat-lto-objects` on targets with linker plugin support.

`-fcompare-elim`

After register allocation and post-register allocation instruction splitting, identify arithmetic instructions that compute processor flags similar to a comparison operation based on that arithmetic. If possible, eliminate the explicit comparison operation.

This pass only applies to certain targets that cannot explicitly represent the comparison operation before register allocation is complete.

Enabled at levels `-O1`, `-O2`, `-O3`, `-Os`.

`-ffold-mem-offsets`

`-fno-fold-mem-offsets`

Try to eliminate add instructions by folding them in memory loads/stores.

Enabled at levels `-O2`, `-O3`.

`-fcprop-registers`

After register allocation and post-register allocation instruction splitting, perform a copy-propagation pass to try to reduce scheduling dependencies and occasionally eliminate the copy.

Enabled at levels `-O1`, `-O2`, `-O3`, `-Os`.

`-fprofile-correction`

Profiles collected using an instrumented binary for multi-threaded programs may be inconsistent due to missed counter updates. When this option is specified, GCC uses heuristics to correct or smooth out such inconsistencies. By default, GCC emits an error message when an inconsistent profile is detected.

This option is enabled by `-fauto-profile`.

`-fprofile-partial-training`

With `-fprofile-use` all portions of programs not executed during training runs are optimized aggressively for size rather than speed. In some cases it is not practical to train all possible hot paths in the program. (For example, it may contain functions specific to a given hardware and training may not cover all hardware configurations the program later runs on.) With `-fprofile-partial-training` profile feedback is ignored for all functions not executed during the training runs, causing them to be optimized as if they were compiled without profile feedback. This leads to better performance when the training is not representative at the cost of significantly bigger code.

`-fprofile-use`

`-fprofile-use=`*path*

Enable profile feedback-directed optimizations, and the following optimizations, many of which are generally profitable only with profile feedback available:

`-fbranch-probabilities` `-fprofile-values`
`-funroll-loops` `-fpeel-loops` `-ftracer` `-fvpt`

```
-finline-functions -fipa-cp -fipa-cp-clone -fipa-bit-cp
-fpredictive-commoning -fsplit-loops -funswitch-loops
-fgcse-after-reload -ftree-loop-vectorize -ftree-slp-vectorize
-fvect-cost-model=dynamic -ftree-loop-distribute-patterns
-fprofile-reorder-functions
```

Before you can use this option, you must first generate profiling information. See [Program Instrumentation Options](#), for information about the `-fprofile-generate` option.

By default, GCC emits an error message if the feedback profiles do not match the source code. This error can be turned into a warning by using `-Wno-error=coverage-mismatch`. Note this may result in poorly optimized code. Additionally, by default, GCC also emits a warning message if the feedback profiles do not exist (see `-Wmissing-profile`).

If *path* is specified, GCC looks at the *path* to find the profile feedback data files. See `-fprofile-dir`.

`-fauto-profile`

`-fauto-profile=`*path*

Enable sampling-based feedback-directed optimizations, and the following optimizations, many of which are generally profitable only with profile feedback available:

```
-fbranch-probabilities -fprofile-values
-funroll-loops -fpeel-loops -ftracer -fvpt
-finline-functions -fipa-cp -fipa-cp-clone -fipa-bit-cp
-fpredictive-commoning -fsplit-loops -funswitch-loops
-fgcse-after-reload -ftree-loop-vectorize -ftree-slp-vectorize
-fvect-cost-model=dynamic -ftree-loop-distribute-patterns
-fprofile-correction
```

path is the name of a file containing AutoFDO profile information. If omitted, it defaults to `fbdata.afdo` in the current directory.

Producing an AutoFDO profile data file requires running your program with the `perf` utility on a supported GNU/Linux target system. For more information, see <https://perfwiki.github.io/main/>.

E.g.

```
perf record -e br_inst_retired:near_taken -b -o perf.data \
-- your_program
```

Then use the `create_gcov` tool to convert the raw profile data to a format that can be used by GCC. You must also supply the unstripped binary for your program to this tool. See <https://github.com/google/autofdo>.

E.g.

```
create_gcov --binary=your_program.unstripped --profile=perf.data \
--gcov=profile.afdo
```

`-fauto-profile-inlining`

When auto-profile is available inline all relevant functions which was inlined in the tran run before reading the profile feedback. This improves context sensitivity of the profile. Enabled by default.

The following options control compiler behavior regarding floating-point arithmetic. These options trade off between speed and correctness. All must be specifically enabled.

`-fexcess-precision=`*style*

This option allows control over excess precision on machines where floating-point operations occur in a format with more precision or range than the IEEE standard and interchange floating-point types. An example of such a target is x87 floating point on x86 processors, which uses an 80-bit representation internally instead of the 64-bit IEEE format. For most programs, the excess precision is harmless, but some programs may rely on the requirements of the C or C++ language standards for handling IEEE values.

By default, `-fexcess-precision=fast` is in effect; this means that operations may be carried out in a wider precision than the types specified in the source if that would result in faster code, and it is unpredictable when rounding to the types specified in the source code takes place. When compiling C or C++, if `-fexcess-precision=standard` is specified then excess precision follows the rules specified in ISO C99 or C++; in particular, both casts and assignments cause values to be rounded to their semantic types (whereas `-ffloat-store` only affects assignments). This option is enabled by default for C or C++ if a strict conformance option such as `-std=c99` or `-std=c++17` is used. `-ffast-math` enables `-fexcess-precision=fast` by default regardless of whether a strict conformance option is used. If `-fexcess-precision=16` is specified, constants and the results of expressions with types `_Float16` and `__bf16` are computed without excess precision.

`-fexcess-precision=standard` is not implemented for languages other than C or C++. On the x86, it has no effect if `-mfpmath=sse` or `-mfpmath=sse+387` is specified; in the former case, IEEE semantics apply without excess precision, and in the latter, rounding is unpredictable.

`-ffloat-store`

Do not store floating-point variables in registers, and inhibit other options that might change whether a floating-point value is taken from a register or memory. This option has generally been subsumed by `-fexcess-precision=standard`, which is more general. If you do use `-ffloat-store`, you may need to modify your program to explicitly store intermediate computations in temporary variables since `-ffloat-store` handles rounding to IEEE format only on assignments and not casts as `-fexcess-precision=standard` does.

`-ffast-math`

Sets the options `-fno-math-errno`, `-funsafe-math-optimizations`, `-ffinite-math-only`, `-fno-rounding-math`, `-fno-signaling-nans`, `-fcx-limited-range` and `-fexcess-precision=fast`.

This option causes the preprocessor macro `__FAST_MATH__` to be defined.

This option is not turned on by any `-O` option besides `-Ofast` since it can result in incorrect output for programs that depend on an exact implementation of IEEE or ISO rules/specifications for math functions. It may, however, yield faster code for programs that do not require the guarantees of these specifications.

`-fno-math-errno`

Do not set `errno` after calling math functions that are executed with a single instruction, e.g., `sqrt`. A program that relies on IEEE exceptions for math error handling may want to use this flag for speed while maintaining IEEE arithmetic compatibility.

This option is not turned on by any `-O` option besides `-Ofast` since it can result in incorrect output for programs that depend on an exact implementation of IEEE or ISO rules/specifications for math functions. It may, however, yield faster code for programs that do not require the guarantees of these specifications.

The default is `-fmath-errno`.

On Darwin systems, the math library never sets `errno`. There is therefore no reason for the compiler to consider the possibility that it might, and `-fno-math-errno` is the default.

`-funsafe-math-optimizations`

Allow optimizations for floating-point arithmetic that (a) assume that arguments and results are valid and (b) may violate IEEE or ANSI standards. When used at link time, it may include libraries or startup files that change the default FPU control word or other similar optimizations.

This option is not turned on by any `-O` option besides `-Ofast` since it can result in incorrect output for programs that depend on an exact implementation of IEEE or ISO rules/specifications for math functions. It may, however, yield faster code for programs that do not require the guarantees of these specifications. Enables `-fno-signed-zeros`, `-fno-trapping-math`, `-fassociative-math` and `-freciprocal-math`.

The default is `-fno-unsafe-math-optimizations`.

`-fassociative-math`

Allow re-association of operands in series of floating-point operations. This violates the ISO C and C++ language standard by possibly changing computation result. NOTE: re-ordering may change the sign of zero as well as ignore NaNs and inhibit or create underflow or overflow (and thus cannot be used on code that relies on rounding behavior like $(x + 2^{*52}) - 2^{*52}$. May also reorder floating-point comparisons and thus may not be used when ordered comparisons are required. This option requires that both `-fno-signed-zeros` and `-fno-trapping-math` be in effect. Moreover, it doesn’t make much sense with `-frounding-math`. For Fortran the option is automatically enabled when both `-fno-signed-zeros` and `-fno-trapping-math` are in effect.

The default is `-fno-associative-math`.

`-freciprocal-math`

Allow the reciprocal of a value to be used instead of dividing by the value if this enables optimizations. For example x / y can be replaced with $x * (1/y)$, which is useful if $(1/y)$ is subject to common subexpression elimination. Note that this loses precision and increases the number of flops operating on the value.

The default is `-fno-reciprocal-math`.

`-ffinite-math-only`

Allow optimizations for floating-point arithmetic that assume that arguments and results are not NaNs or +-Infs.

This option is not turned on by any `-O` option besides `-Ofast` since it can result in incorrect output for programs that depend on an exact implementation of IEEE or ISO rules/specifications for math functions. It may, however, yield faster code for programs that do not require the guarantees of these specifications.

The default is `-fno-finite-math-only`.

`-fno-signed-zeros`

Allow optimizations for floating-point arithmetic that ignore the signedness of zero. IEEE arithmetic specifies the behavior of distinct $+0.0$ and -0.0 values, which then prohibits simplification of expressions such as $x+0.0$ or $0.0*x$ (even with `-ffinite-math-only`). This option implies that the sign of a zero result isn’t significant.

The default is `-fsigned-zeros`.

`-fno-trapping-math`

Compile code assuming that floating-point operations cannot generate user-visible traps. These traps include division by zero, overflow, underflow, inexact result and invalid operation. This option requires that `-fno-signaling-nans` be in effect. Setting this option may allow faster code if one relies on “non-stop” IEEE arithmetic, for example.

This option is not turned on by any `-O` option besides `-Ofast` since it can result in incorrect output for programs that depend on an exact implementation of IEEE or ISO rules/specifications for math functions.

The default is `-ftrapping-math`.

Future versions of GCC may provide finer control of this setting using C99’s `FENV_ACCESS` pragma. This command-line option will be used along with `-frounding-math` to specify the default state for `FENV_ACCESS`.

`-frounding-math`

Disable transformations and optimizations that assume default floating-point rounding behavior (round-to-nearest). This option should be specified for programs that change the FP rounding mode dynamically, or that may be executed with a non-default rounding mode. This option disables constant folding of floating-point expressions at compile time (which may be affected by rounding mode) and arithmetic transformations that are unsafe in the presence of sign-dependent rounding modes.

The default is `-fno-rounding-math`.

This option is experimental and does not currently guarantee to disable all GCC optimizations that are affected by rounding mode. Future versions of GCC may provide finer control of this setting using C99’s `FENV_ACCESS` pragma. This command-line option will be used along with `-ftrapping-math` to specify the default state for `FENV_ACCESS`.

`-fsignaling-nans`

Compile code assuming that IEEE signaling NaNs may generate user-visible traps during floating-point operations. Setting this option disables optimizations that may change the number of exceptions visible with signaling NaNs. This option implies `-ftrapping-math`.

This option causes the preprocessor macro `__SUPPORT_SNAN__` to be defined.

The default is `-fno-signaling-nans`.

This option is experimental and does not currently guarantee to disable all GCC optimizations that affect signaling NaN behavior.

`-fsingle-precision-constant`

Treat floating-point constants as single precision instead of implicitly converting them to double-precision constants.

`-fcx-limited-range`

When enabled, this option states that a range reduction step is not needed when performing complex division. Also, there is no checking whether the result of a complex multiplication or division is $\text{NaN} + I*\text{NaN}$, with an attempt to rescue the situation in that case. The option is enabled by `-ffast-math`.

This option controls the default setting of the ISO C99 `CX_LIMITED_RANGE` pragma. Nevertheless, the option applies to all languages.

`-fcx-fortran-rules`

Complex multiplication and division follow Fortran rules. Range reduction is done as part of complex division, but there is no checking whether the result of a complex multiplication or division is $\text{NaN} + I*\text{NaN}$, with an attempt to rescue the situation in that case.

`-fcx-method=method`

Complex multiplication and division follow the stated *method*. The *method* argument should be one of ‘limited-range’, ‘fortran’ or ‘stdc’.

The default is to honor language specific constraints which means ‘fortran’ for Fortran and ‘stdc’ otherwise.

The following options control optimizations that may improve performance, but are not enabled by any `-O` options. This section includes experimental options that may produce broken code.

`-fbranch-probabilities`

After running a program compiled with `-fprofile-arcs` (see [Program Instrumentation Options](#)), you can compile it a second time using `-fbranch-probabilities`, to improve optimizations based on the number of times each branch was taken. When a program compiled with `-fprofile-arcs` exits, it saves arc execution counts to a file called *sourcename.gcda* for each source file. The information in this data file is very dependent on the structure of the generated code, so you must use the same source code and the same optimization options for both compilations. See details about the file naming in `-fprofile-arcs`.

With `-fbranch-probabilities`, GCC puts a ‘`REG_BR_PROB`’ note on each ‘`JUMP_INSN`’ and ‘`CALL_INSN`’. These can be used to improve optimization. Currently, they are only used in one place: in `reorg.cc`, instead of guessing which path a branch is most likely to take, the ‘`REG_BR_PROB`’ values are used to exactly determine which path is taken more often.

Enabled by `-fprofile-use` and `-fauto-profile`.

`-fprofile-values`

If combined with `-fprofile-arcs`, it adds code so that some data about values of expressions in the program is gathered.

With `-fbranch-probabilities`, it reads back the data gathered from profiling values of expressions for usage in optimizations.

Enabled by `-fprofile-generate`, `-fprofile-use`, and `-fauto-profile`.

`-fprofile-reorder-functions`

Function reordering based on profile instrumentation collects first time of execution of a function and orders these functions in ascending order, aiming to optimize program startup through more efficient loading of text segments.

Enabled with `-fprofile-use`.

`-fvpt`

If combined with `-fprofile-arcs`, this option instructs the compiler to add code to gather information about values of expressions.

With `-fbranch-probabilities`, it reads back the data gathered and actually performs the optimizations based on them. Currently the optimizations include specialization of division operations using the knowledge about the value of the denominator.

Enabled with `-fprofile-use` and `-fauto-profile`.

`-frename-registers`

Attempt to avoid false dependencies in scheduled code by making use of registers left over after register allocation. This optimization most benefits processors with lots of registers. Depending on the debug information format adopted by the target, however, it can make debugging impossible, since variables no longer stay in a “home register”.

Enabled by default with `-funroll-loops`.

`-fschedule-fusion`

Performs a target dependent pass over the instruction stream to schedule instructions of same type together because target machine can execute them more efficiently if they are adjacent to each other in the instruction flow.

Enabled at levels `-O2`, `-O3`, `-Os`.

`-fdep-fusion`

Detect macro-op fusible pairs consisting of single-use instructions and their uses, and place such pairs together in the instruction stream to increase fusion opportunities in hardware. This pass is executed once before register allocation, and another time before register renaming.

Enabled at levels `-O2`, `-O3`, `-Os`.

`-ftracer`

Perform tail duplication to enlarge superblock size. This transformation simplifies the control flow of the function allowing other optimizations to do a better job.

Enabled by `-fprofile-use` and `-fauto-profile`.

`-funroll-loops`

Unroll loops whose number of iterations can be determined at compile time or upon entry to the loop. `-funroll-loops` implies `-frerun-cse-after-loop`, `-fweb` and `-frename-registers`. It also turns on complete loop peeling (i.e. complete removal of loops with a small constant number of iterations). This option makes code larger, and may or may not make it run faster.

Enabled by `-fprofile-use` and `-fauto-profile`.

`-funroll-all-loops`

Unroll all loops, even if their number of iterations is uncertain when the loop is entered. This usually makes programs run more slowly. `-funroll-all-loops` implies the same options as `-funroll-loops`.

`-fpeel-loops`

Peels loops for which there is enough information that they do not roll much (from profile feedback or static analysis). It also turns on complete loop peeling (i.e. complete removal of loops with small constant number of iterations).

Enabled by `-O3`, `-fprofile-use`, and `-fauto-profile`.

`-fmalloc-dce`

Control whether `malloc` (and its variants such as `calloc` or `strdup`), can be optimized away provided its return value is only used as a parameter of `free` call or compared with `NULL`. If `-fmalloc-dce=1` is used, only calls to `free` are allowed while with `-fmalloc-dce=2` also comparisons with `NULL` pointer are considered safe to remove.

The default is `-fmalloc-dce=2`. See also `-fallocation-dce`.

`-fmove-loop-invariants`

Enables the loop invariant motion pass in the RTL loop optimizer. Enabled at level `-O1` and higher, except for `-Og`.

`-fmove-loop-stores`

Enables the loop store motion pass in the GIMPLE loop optimizer. This moves invariant stores to after the end of the loop in exchange for carrying the stored value in a register across the iteration. Note for this option to have an effect `-ftree-loop-im` has to be enabled as well. Enabled at level `-O1` and higher, except for `-Og`.

`-fsplit-loops`

Split a loop into two if it contains a condition that’s always true for one side of the iteration space and false for the other.

Enabled by `-fprofile-use` and `-fauto-profile`.

`-funswitch-loops`

Move branches with loop invariant conditions out of the loop, with duplicates of the loop on both branches (modified according to result of the condition).

Enabled by `-fprofile-use` and `-fauto-profile`.

`-fversion-loops-for-strides`

If a loop iterates over an array with a variable stride, create another version of the loop that assumes the stride is always one. For example:

```
for (int i = 0; i < n; ++i)
  x[i * stride] = ...;
```

becomes:

```
if (stride == 1)
  for (int i = 0; i < n; ++i)
    x[i] = ...;
else
  for (int i = 0; i < n; ++i)
    x[i * stride] = ...;
```

This is particularly useful for assumed-shape arrays in Fortran where (for example) it allows better vectorization assuming contiguous accesses. This flag is enabled by default at -O3. It is also enabled by -fprofile-use and -fauto-profile.

-ffunction-sections
-fdata-sections

Place each function or data item into its own section in the output file if the target supports arbitrary sections. The name of the function or the name of the data item determines the section’s name in the output file.

Use these options on systems where the linker can perform optimizations to improve locality of reference in the instruction space. Most systems using the ELF object format have linkers with such optimizations. On AIX, the linker rearranges sections (CSECTs) based on the call graph. The performance impact varies.

Together with a linker garbage collection (linker --gc-sections option) these options may lead to smaller statically-linked executables (after stripping).

On ELF/DWARF systems these options do not degenerate the quality of the debug information. There could be issues with other object files/debug info formats.

Only use these options when there are significant benefits from doing so. When you specify these options, the assembler and linker create larger object and executable files and are also slower. These options affect code generation. They prevent optimizations by the compiler and assembler using relative locations inside a translation unit since the locations are unknown until link time. An example of such an optimization is relaxing calls to short call instructions.

-fstdarg-opt

Optimize the prologue of variadic argument functions with respect to usage of those arguments.

-fsection-anchors

Try to reduce the number of symbolic address calculations by using shared “anchor” symbols to address nearby objects. This transformation can help to reduce the number of GOT entries and GOT accesses on some targets.

For example, the implementation of the following function foo:

```
static int a, b, c;
int foo (void) { return a + b + c; }
```

usually calculates the addresses of all three variables, but if you compile it with -fsection-anchors, it accesses the variables from a common anchor point instead. The effect is similar to the following pseudocode (which isn’t valid C):

```
int foo (void)
{
  register int *xr = &x;
  return xr[&a - &x] + xr[&b - &x] + xr[&c - &x];
}
```

Not all targets support this option.

-fzero-call-used-regs=*choice*

Zero call-used registers at function return to increase program security by either mitigating Return-Oriented Programming (ROP) attacks or preventing information leakage through registers.

The possible values of *choice* are the same as for the zero_call_used_regs attribute (see [Common Attributes](#)). The default is ‘skip’.

You can control this behavior for a specific function by using the function attribute zero_call_used_regs (see [Common Attributes](#)).

Next: [Program Instrumentation Options](#), Previous: [Options for Debugging Your Program](#), Up: [GCC Command Options](#) [[Contents](#)][[Index](#)]